Farmers Market Database: SQL Practice Questions

Topics: SELECT · WHERE · AND / OR / NOT · IN · BETWEEN · LIKE · CONCAT · INSTR · LEFT · RIGHT · SUBSTRING · TRIM · LENGTH · REVERSE · ROUND · DISTINCT · Derived Columns · CASE

SQL flavour: MySQL 8

Rules for all questions

- Every question uses **one table only**. You don't need JOINs, GROUP BY, aggregates or subqueries.
- You may use `ORDER BY`, `LIMIT`, column aliases, arithmetic operators (`+ - * / % DIV`), `UPPER`, `LOWER`, `REPLACE` and `REPEAT` as helpers.
- Give every derived column a clear alias.
- For "What will this return?" questions, work out the answer on paper first, then run the query to check.

Schema Reference

Table	Columns
`customer`	customer_id, customer_first_name, customer_last_name, customer_zip *(VARCHAR)*
`vendor`	vendor_id, vendor_name, vendor_type, vendor_owner_first_name, vendor_owner_last_name
`booth`	booth_number, booth_price, booth_description, booth_size *(e.g. '10x10')*
`product`	product_id, product_name, product_category_id, product_sku, product_size
`product_category`	product_category_id, product_category_name
`market_date_info`	market_date, market_day, market_week *(VARCHAR)*, market_year *(VARCHAR)*, market_season, market_temp_min, market_temp_max, market_rain_flag, market_snow_flag
`vendor_inventory`	market_date, vendor_id, product_id, quantity, original_price
`customer_purchases`	product_id, vendor_id, market_date, customer_id, quantity, unit_customer_price, transaction_time
`vendor_booth_assignments`	vendor_id, booth_number, market_date

Temperatures are in °F and prices are in $.

Section 1: SELECT & Derived Columns

Q1. From `customer_purchases`, show `product_id`, `customer_id`, `quantity`, `unit_customer_price` and a derived column `total_amount` (quantity × unit price).

Q2. From `customer_purchases`, show each transaction's `total_amount` and `amount_with_tax`, assuming 8% sales tax. Round the tax-inclusive amount to 2 decimal places.

Q3. From `vendor_inventory`, calculate `inventory_value` (quantity × original_price) for every row. Show the rows from highest to lowest inventory value.

Q4. From `market_date_info`, show each `market_date` with its minimum and maximum temperature, a `temp_range` column (max − min) and an `avg_temp` column (the midpoint of min and max).

Q5. Convert `market_temp_min` and `market_temp_max` from Fahrenheit to Celsius using `(F - 32) × 5/9`. Show them as `temp_min_c` and `temp_max_c`, rounded to 1 decimal place.

Q6. From `booth`, show `booth_number`, `booth_price`, `monthly_cost` (booth rented for 4 market days a month) and `annual_cost` (52 market days a year).

Q7. The market plans a 15% markup on all inventory. From `vendor_inventory`, show `original_price`, `new_price` and `price_increase` (the difference between the two). Round both derived columns to 2 decimals.

Q8. A flat $2.50 discount is applied to every unit sold. From `customer_purchases`, show `unit_customer_price`, `discounted_unit_price` and `total_after_discount`.

Q9. From `customer_purchases`, return the columns in this order: customer_id, market_date, transaction_time, product_id and the bill total. Use the aliases **"Customer ID"** and **"Bill Amount"**, which contain spaces.

Q10. Can you use an alias defined in the SELECT list inside the WHERE clause of the same query? Write a correct query on `customer_purchases` that returns only transactions whose total (quantity × unit price) is above 400, and show the total as a derived column.

Q11. From `vendor_inventory`, calculate `expected_revenue`, assuming each vendor sells 80% of its stock at the original price. Show only the top 5 rows by expected revenue.

Q12. From `booth`, show `booth_price`, `price_per_day` (price covers a 3-day event, rounded to 2 decimals) and `diff_from_base` (booth_price − 100).

Q13. From `market_date_info`, show each `market_date` with a column `is_freezing` defined as the expression `market_temp_min < 32` (no CASE). What values does MySQL show in this column, and why?

Q14. From `market_date_info`, create a derived column `weather_events` = `market_rain_flag + market_snow_flag`. Return only the dates where this value is greater than 0.

Q15. From `customer_purchases`, show the original bill total, the total if each customer had bought **one unit fewer**, and the `revenue_loss` between the two.

Q16. From `vendor`, show `vendor_id`, `vendor_name` and a constant column `market_name` with the value `'Green Valley Farmers Market'` for every row.

Q17. From `customer`, show all columns plus two derived columns: `country` (always `'USA'`) and `loyalty_points` (customer_id × 10).

Q18. From `vendor_inventory`, show `quantity`, `full_dozens` (whole dozens available, using `DIV`) and `loose_units` (the units left over, using `%`).

Q19. Compare `quantity + 1 * unit_customer_price` with `(quantity + 1) * unit_customer_price` on `customer_purchases`. Show both as columns and explain why they differ.

Q20. In `market_date_info`, `market_year` and `market_week` are VARCHAR columns. Create a derived column `week_code = market_year * 100 + market_week` (e.g. 202401). Does the arithmetic work on string columns? What does MySQL do behind the scenes?

Q21. From `booth`, show `booth_price`, `gst_amount` (18% of the price) and `final_price` (price + GST), all rounded to 2 decimals.

Q22. Write a query on `vendor` that returns every column but puts `vendor_id` as the **last** column. Why do interviewers prefer explicit column lists to `SELECT *` in production code?

Section 2: Filtering with WHERE

Q23. Find all customers whose zip code is `'12345'`.

Q24. `customer_zip` is VARCHAR. Do `WHERE customer_zip = 12345` and `WHERE customer_zip = '12345'` return the same rows? Now compare `WHERE customer_zip = 01234` with `WHERE customer_zip = '01234'`. Explain what happens in each case.

Q25. From `vendor_inventory`, find the rows where `original_price` is at least 25.

Q26. Find all purchases made on `'2024-01-15'`.

Q27. Find all purchases whose total (quantity × unit price) exceeds 500, without using a SELECT alias in the WHERE clause.

Q28. Find the market dates where the temperature range (max − min) is more than 15 degrees.

Q29. List the market dates on which it snowed.

Q30. Find the booths whose price is **not** 150. Write the condition twice, once with `!=` and once with `<>`.

Q31. Find the purchases made after 6 PM (`transaction_time` later than `'18:00:00'`).

Q32. Run `WHERE product_size = 'large'` (lowercase) on `product`. Does it return the 'Large' products? What decides whether string comparison is case-sensitive in MySQL?

Q33. Data check: find the products whose `product_id` is different from `product_category_id`.

Q34. Find the market dates where the minimum temperature was below freezing (32 °F).

Q35. From `vendor_inventory`, find the rows where inventory value (quantity × original_price) exceeds 3000.

Q36. Find the customers with an even `customer_id`, using the `%` operator.

Q37. From `customer_purchases`, find the transactions where `unit_customer_price` is more than twice the `quantity`.

Q38. On `market_date_info`, compare `WHERE market_week = 2` with `WHERE market_week = '2'`. Do both work? Which one is better, and why?

Q39. Find the purchases where `market_date > '2024-01-10'`. How does MySQL compare a DATE column with a string literal?

Q40. What is wrong with `SELECT * FROM customer WHERE customer_zip = NULL;`? Rewrite it correctly and explain why `= NULL` never matches.

Section 3: AND / OR / NOT

Q41. Find the vendors of `'Type B'` whose owner's last name is `'White'`.

Q42. Find the Winter market dates that had snow **and** a minimum temperature below 30.

Q43. Find the market dates that had rain **or** snow.

Q44. Find the purchases where quantity is more than 10 **and** unit price is below 35.

Q45. Find the booths that are either a `'Large booth'` or priced above 180.

Q46. What will this query return?

```
SELECT * FROM vendor
WHERE vendor_type = 'Type A' OR vendor_type = 'Type B' AND vendor_id > 15;
```

Rewrite it with parentheses so that it returns only vendors with `vendor_id > 15`, of either type.

Q47. Find the customers whose zip is `'12345'` or `'23456'` and whose `customer_id` is greater than 10. Watch the operator precedence.

Q48. Using `NOT` and `OR` (no IN), list the market dates that are **not** Saturday or Sunday.

Q49. Rewrite `WHERE NOT (quantity < 50 OR original_price > 30)` on `vendor_inventory` **without** using NOT, applying De Morgan's law. Check that both versions return the same rows.

Q50. Using only comparison operators and AND (no BETWEEN), find the purchases made from 10:00 AM to 4:00 PM (inclusive) with quantity of at least 5.

Q51. Find the products that are 'Small' or 'Medium' **and** belong to a category id greater than 5.

Q52. Find the market dates where the minimum temperature was below 30 or the maximum was above 48, **but** there was no snow.

Q53. Find the customers with `customer_id` greater than 5 and less than 15 whose zip is not `'45678'`.

Q54. Find the booths that are either Medium booths priced over 150, or Small booths priced under 130.

Q55. Find the purchases where (quantity ≥ 15 and unit price ≥ 35) **or** quantity ≤ 3.

Q56. Compare these two filters on `market_date_info`:

- `WHERE NOT market_rain_flag = 1 AND NOT market_snow_flag = 1`
- `WHERE NOT (market_rain_flag = 1 AND market_snow_flag = 1)`

Do they return the same dates? Explain the difference.

Q57. Find the vendors where **exactly one** of these is true: the owner's last name is `'White'`, or the vendor type is `'Type A'`. Rows where both are true should not appear.

Q58. Find the Tuesdays or Thursdays in market week 2 where the maximum temperature was at least 45.

Q59. From `vendor_inventory`, find the rows for vendor ids 5 to 10 (inclusive) where the original price is not 20. Use only AND/NOT and comparison operators.

Q60. Find the purchases that are either morning purchases (before 12:00) with a total above 100, or evening purchases (after 18:00) with a total above 300.

Q61. What will this return?

```
SELECT NULL AND 0, NULL AND 1, NULL OR 1, NULL OR 0, NOT NULL;
```

Q62. Find the booths that are **not** Large booths **and** have an odd booth number.

Section 4: IN / NOT IN

Q63. Find the customers whose zip code is `'12345'`, `'56789'` or `'90123'`.

Q64. Using IN, list the market dates that fall on a Saturday or Sunday.

Q65. Find the vendors whose owner's last name is Smith, Brown, White or Jones.

Q66. Find the products whose size is **not** 'Small' or 'Large'.

Q67. Find all purchases made on `'2024-01-03'`, `'2024-01-09'`, `'2024-01-14'` or `'2024-01-20'`.

Q68. From `vendor_inventory`, find the rows where `product_id` is not 1, 5, 10, 15 or 20.

Q69. Find the booths whose size is `'10x10'` or `'6x6'`, excluding booth numbers 1, 3 and 19.

Q70. What will this return, and why?

```
SELECT * FROM customer WHERE customer_id NOT IN (1, 2, NULL);
```

Q71. What will this return? Compare it with Q70 and explain the difference.

```
SELECT * FROM customer WHERE customer_id IN (1, 2, NULL);
```

Q72. Find the customers whose zip code **starts** with 1, 2 or 3, using LEFT together with IN.

Q73. Find the weekday market dates (Monday to Friday, using IN) on which it snowed.

Q74. Find the vendors whose owner's first name is Alice, Bob, Eve or Grace. Show the owner's full name as one column.

Q75. Rewrite this with IN, then say whether the two versions differ in result or readability:

```
WHERE market_day = 'Monday' OR market_day = 'Wednesday' OR market_day = 'Friday'
```

Q76. Find the purchases whose `transaction_time` is `'10:00:00'`, `'12:00:00'` or `'14:00:00'`. Would the query still work if the values were written as `'10:00'`, `'12:00'`, `'14:00'`?

Q77. Find the products in categories 1, 3, 5, 7 or 9 whose size is Medium or Large.

Q78. Find the customers whose zip code **ends** in 0 or 5, using RIGHT with IN.

Q79. `market_week` is VARCHAR. Does `WHERE market_week IN (1, 3)` work? What type conversion happens?

Q80. Find the booths whose description is 'Large booth' or 'Small booth' and whose price is neither 100 nor 190.

Section 5: BETWEEN

Q81. Find all purchases between `'2024-01-05'` and `'2024-01-10'`. Are both end dates included? How many rows do you expect?

Q82. Find the booths priced between 120 and 160.

Q83. From `vendor_inventory`, find the rows where quantity is **not** between 50 and 150.

Q84. What will `SELECT * FROM booth WHERE booth_price BETWEEN 160 AND 120;` return? Explain.

Q85. Find the purchases made between 9 AM and 5 PM using BETWEEN on `transaction_time`.

Q86. The market wants the "late-night" transactions from 10 PM to 4 AM. Why does `transaction_time BETWEEN '22:00:00' AND '04:00:00'` fail? Write a query that works.

Q87. Does `WHERE customer_last_name BETWEEN 'A' AND 'H'` include 'Harris'? Explain why, and fix the query so that all last names starting with A to H are included.

Q88. What will this return, and why?

```
SELECT DISTINCT market_week FROM market_date_info
WHERE market_week BETWEEN '1' AND '10';
```

Q89. Find the purchases whose total (quantity × unit price) is between 200 and 500.

Q90. Find the market dates where the temperature range (max − min) is between 14 and 15.

Q91. From `vendor_inventory`, find the rows with original price between 20 and 30 **and** market date between `'2024-01-01'` and `'2024-01-12'`.

Q92. Suppose `market_date` were a DATETIME column holding values like `'2024-01-10 15:30:00'`. Which rows would `BETWEEN '2024-01-05' AND '2024-01-10'` miss? Write a safer version of the filter.

Q93. Find the customers with `customer_id` between 5 and 15, excluding customer 10.

Q94. What will this return, and in which order would the names sort?

```
SELECT product_name FROM product
WHERE product_name BETWEEN 'Product 1' AND 'Product 2';
```

Q95. Find the booths numbered 5 to 15 whose price is **not** between 130 and 150.

Q96. Find the market dates where the average of min and max temperature is between 38 and 40.

Q97. Using string comparison, find the customers whose zip is between `'20000'` and `'60000'`. Where would `'01234'` fall, and why?

Q98. Using RIGHT on `market_date` together with BETWEEN, find the market dates that fall in the first half of the month (day 1 to 15).

Section 6: Pattern Matching with LIKE

Q99. Find the customers whose first name starts with a vowel.

Q100. Find the customers whose last name ends with `'son'`.

Q101. Using the `_` wildcard, find the customers whose first name has exactly 3 characters.

Q102. Find the customers whose first name has `'a'` as its second letter.

Q103. Find the customers whose last name contains `'o'` but does **not** start with `'O'`.

Q104. Work out how many rows each condition returns on `product`, then check:

- `product_name LIKE '%1%'`
- `product_name LIKE '%1'`
- `product_name LIKE 'Product 1_'`

Q105. Using LIKE only, find the products whose name ends with a two-digit number.

Q106. Find the product SKUs that are exactly 4 characters long, using LIKE only.

Q107. Find the market days whose name contains `'es'`.

Q108. Find the booths whose size starts with `'1'`.

Q109. Find the customers whose zip code starts with `'0'`. Why is the leading zero kept?

Q110. What does `WHERE vendor_name LIKE '%_%'` return, and why? If some vendor names contained a real underscore (e.g. `'Vendor_21'`), how would you search for the literal `_`?

Q111. Find the vendors whose owner's first name does **not** start with a vowel.

Q112. Find the customers whose last name contains a double letter (`ll` , `rr` , `oo` , `ee` , `ss` , `nn` or `tt`).

Q113. Does `WHERE customer_first_name LIKE 'a%'` return 'Alice'? How would you make the match case-sensitive?

Q114. Find the customers whose first name ends with `'a'` .

Q115. Find the customers whose first name starts with a consonant **and** ends with a vowel.

Q116. Using LIKE on `transaction_time` , find the purchases made from 10:00 to 19:59. What does MySQL do when LIKE is used on a TIME column?

Q117. Using LIKE on `market_date` , find the market dates from the 10th to the 19th of the month.

Q118. Find the vendors whose owner's full name (first + last, joined inside the query) contains `'an'` .

Q119. Find the customers whose last name has `'r'` as the **third-last** letter.

Q120. Compare `vendor_name = 'Vendor 1 '` with `vendor_name LIKE 'Vendor 1 '` (note the trailing space). Do they return the same rows? Explain how trailing spaces are handled by `=` and by `LIKE` .

Q121. Find the products whose name contains neither `'1'` nor `'2'` .

Q122. Find the customers whose first name has exactly 5 letters and contains `'i'` .

Section 7: CONCAT

Q123. Show each customer's name as **"Last, First"** (e.g. `Johnson, Alice`).

Q124. Show vendor details as **"John Doe (Vendor 1)"**.

Q125. Create a customer label in the format **"Alice Johnson - 12345"**.

Q126. Generate an email ID for every customer in the format `firstname.lastname@farmersmarket.com` , all in lowercase.

Q127. Create a booth label in the format **"#1 | Large booth | 10x10 | $100.00"**.

Q128. Create a vendor code in the format **"A-001"**: the type letter from `vendor_type` , a hyphen, then `vendor_id` zero-padded to 3 digits. Don't use LPAD. Hint: CONCAT and RIGHT.

Q129. From `market_date_info` , create the sentence **"Monday, 2024-01-01: 30°F to 45°F"** for every date.

Q130. What will `SELECT CONCAT('Vendor', NULL, 'X');` return? How would you protect a CONCAT against NULL values?

Q131. From `customer_purchases` , create a description such as **"2 units @ $20.00 = $40.00"**.

Q132. Show customer initials in the format **"A.J."**.

Q133. Create a product display string: **"Product 1 [SKU1] - Small"**.

Q134. What does `SELECT 'Farm' || 'Market';` return in MySQL by default? Why is `||` risky for string joining in MySQL, compared with other databases?

Q135. Create an `inventory_key` for each `vendor_inventory` row by joining `market_date`, `vendor_id` and `product_id` with underscores (e.g. `2024-01-01_1_1`).

Q136. Find the customer whose full name, built with CONCAT inside the WHERE clause, is `'Grace Davis'`. Why can this be slower than filtering on the two columns separately?

Section 8: INSTR, LEFT, RIGHT, SUBSTRING

Q137. Show the first 3 letters of each customer's first name in uppercase, as `short_code`.

Q138. Show the last 2 characters of each customer's zip code.

Q139. From `market_date_info`, show `year_month` (e.g. `2024-01`) using LEFT and `day_of_month` using RIGHT.

Q140. Show each `market_day` as a 3-letter uppercase abbreviation (`MON`, `TUE`, ...).

Q141. Using INSTR, find the position of `'x'` in each `booth_size`.

Q142. Split `booth_size` into `booth_width` and `booth_length`. Don't hard-code positions, because the sizes have different lengths (`'10x10'` vs `'8x8'`).

Q143. Using the width and length from Q142, calculate each booth's `area` and its `price_per_sq_ft` (rounded to 2 decimals).

Q144. Extract the number from `product_name` (e.g. `'Product 12'` → `12`), using INSTR to locate the space.

Q145. Extract the number from `product_sku` (text after `'SKU'`) and return only the rows where that number does **not** match `product_id`.

Q146. From `vendor`, extract the vendor number from `vendor_name` and the type letter (`A` / `B`) from `vendor_type`.

Q147. Extract the first word of `booth_description` (e.g. `'Large'`), using INSTR.

Q148. Using INSTR (not LIKE), find the customers whose first name does **not** contain the letter `'e'`.

Q149. Find the customers whose last name contains `'a'` somewhere **after** the 3rd position.

Q150. Compare `SUBSTRING(customer_last_name, -3)` with `RIGHT(customer_last_name, 3)`. Are the results the same?

Q151. From `customer_purchases`, extract the month from `market_date` using SUBSTRING, and the hour and minute from `transaction_time` as separate columns.

Q152. Show the 2nd to 4th letters of each customer's first name.

Q153. Find the customers whose first name starts with a letter that comes **later** in the alphabet than the first letter of their last name (e.g. Tom Robinson: T > R).

Q154. Mask customer zip codes so that only the first 3 digits show: `'12345'` → `'123**'`.

Q155. Mask each customer's last name so that only the first letter shows and the rest are `*` (e.g. `Johnson` → `J******`). The number of stars must match the name length.

Q156. What do these return in MySQL? Explain any surprising results.

```
SELECT SUBSTRING('Farmers', 1, 3), SUBSTRING('Farmers', 0, 3), SUBSTRING('Farmers', 4),
SUBSTRING('Farmers', -4, 2);
```

Q157. INSTR only finds the first occurrence. Find the position of the **second** `'o'` in each customer's last name, and show only names that have a second `'o'`.

Q158. From `market_date_info`, extract the year from `market_date` using LEFT, and return any rows where it does not match the `market_year` column.

Q159. Using RIGHT instead of LIKE, find the vendors whose owner's last name ends with `'son'`.

Q160. Create a product code of the form `S-1`, `M-2`, `L-3`: the first letter of `product_size`, a hyphen, then `product_id`.

Q161. Find the transactions whose `transaction_time` is exactly on the hour (minutes and seconds are `00:00`), using RIGHT.

Q162. Using SUBSTRING and INSTR (no hard-coded positions), extract the text that comes after `'Type '` in `vendor_type`.

Q163. Find the customers whose zip code has `'5'` as its **third** digit.

Q164. From `vendor`, show the owner's initials (e.g. `JD`) and return only the vendors where **both** initials fall in the first half of the alphabet (A–M).

Section 9: TRIM, LENGTH, REVERSE

Q165. Show each customer's full name and its length (including the space), longest first.

Q166. Find the customers whose first and last names have the same number of characters.

Q167. What will this return?

```
SELECT LENGTH('  Vendor 1  '), LENGTH(TRIM('  Vendor 1  ')),
       LENGTH(LTRIM('  Vendor 1  ')), LENGTH(RTRIM('  Vendor 1  '));
```

Q168. Use `TRIM(LEADING '0' FROM customer_zip)` and return only the customers whose zip value changes as a result.

Q169. Use `TRIM(TRAILING ' booth' FROM booth_description)` to get just the size word (`Large`, `Medium`, `Small`).

Q170. Using TRIM with LEADING (not SUBSTRING), extract the numeric part of `product_sku` and of `product_name`.

Q171. What will `TRIM(TRAILING 'day' FROM market_day)` return for each weekday? Pay attention to Tuesday, Wednesday and Thursday.

Q172. What is the difference between `LENGTH('Café')` and `CHAR_LENGTH('Café')` in MySQL? Why does it matter for names with accented characters?

Q173. Using REVERSE, find the customers and the vendor owners whose first name is a palindrome (reads the same backwards, ignoring case). Write one query per table.

Q174. Extract the **last word** of `product_name` using REVERSE and INSTR, without assuming the word's length.

Q175. Find the position of the **last** `'o'` in each customer's last name, using REVERSE, INSTR and LENGTH.

Q176. Find the customers whose last name is at least 3 characters longer than their first name.

Q177. Count how many times the letter `'a'` (upper or lower case) appears in each customer's full name, using LENGTH and REPLACE. Show only customers with 2 or more.

Q178. Count the words in each `booth_description`, using LENGTH and REPLACE.

Q179. Using LENGTH instead of LIKE, find the vendors whose name has a two-digit number (e.g. `'Vendor 10'`).

Q180. What will `SELECT TRIM(BOTH '0' FROM '00120400');` return?

Q181. What does `REVERSE(booth_price)` return for a booth priced at 100.00? What does this tell you about how REVERSE handles numbers?

Q182. Using REVERSE and LEFT only (no RIGHT, no LIKE), find the customers whose first name ends with `'e'`.

Section 10: ROUND & Numeric Formatting

Q183. From `market_date_info`, convert the average temperature (midpoint of min and max) to Celsius and show it rounded to 0 decimals and to 1 decimal, side by side.

Q184. Round each `booth_price` to the nearest 10 and to the nearest 100. What happens for prices of 145 and 155?

Q185. What will this return in MySQL?

```
SELECT ROUND(2.5), ROUND(3.5), ROUND(-2.5), ROUND(2.45, 1), ROUND(1234.5678, -2);
```

Q186. From `customer_purchases`, calculate the total with 7.25% tax, both rounded and truncated to 2 decimals (`ROUND` vs `TRUNCATE`). Show a column with the difference between the two.

Q187. From `vendor_inventory`, apply a 12.5% discount to `original_price` and round the result to the **nearest 5 cents** (0.05).

Q188. Without using CEIL/CEILING, round the discounted price from Q187 **up** to the next whole dollar.

Q189. Show each booth's price as a percentage of the top price (195). Round to 1 decimal and add a `%` sign, e.g. `51.3%`.

Q190. From `customer_purchases`, apply three successive 10% discounts to `unit_customer_price` and round to 2 decimals. Is that the same as a single 30% discount?

Q191. Find the purchases whose total, including 8% tax and rounded to the nearest dollar, is more than 500.

Q192. What will `SELECT ROUND('12.567', 1), ROUND('12.567abc', 1);` return? Explain.

Q193. From `market_date_info`, show the temperature range as a percentage of the maximum temperature, rounded to 1 decimal.

Q194. A vendor packs produce in boxes of 12. From `vendor_inventory`, show `ROUND(quantity / 12)` as boxes needed. Why is ROUND the wrong function here? Give a correct expression.

Section 11: DISTINCT

Q195. List the distinct booth sizes.

Q196. List the distinct `(booth_description, booth_size)` pairs. What does the result tell you about how consistent the booth data is?

Q197. List the distinct customer zip codes in ascending order. Why does `'01234'` appear first?

Q198. Explain what `SELECT DISTINCT market_season, market_day FROM market_date_info;` returns. Does DISTINCT apply to one column or to both?

Q199. List the distinct first letters of customer last names, sorted alphabetically.

Q200. List the distinct year-month values (`'2024-01'`) from `market_date_info`, using LEFT.

Q201. List the distinct vendor owner last names. Which last name(s) appear more than once in the table? (Find out by comparing with the full list; don't use GROUP BY.)

Q202. Does `SELECT DISTINCT(vendor_type), vendor_name FROM vendor;` return only distinct vendor types? Explain what the parentheses actually do.

Q203. List the distinct product sizes, excluding 'Small'.

Q204. Using CASE, label each market date as 'Snow', 'Rain' or 'Dry', and list the distinct labels.

Q205. If a column contains three NULL values, how many NULL rows will `SELECT DISTINCT` on that column return?

Q206. Does this query run in MySQL 8? Explain the error or the result.

```
SELECT DISTINCT market_day FROM market_date_info ORDER BY market_date;
```

Q207. List the distinct combinations of `market_week` and `market_day` on which it snowed.

Q208. List the distinct temperature ranges (max − min) from `market_date_info`, sorted from smallest to largest.

Section 12: CASE Statements

Q209. Classify booths by price: `'Premium'` (≥ 175), `'Standard'` (130 to 174.99), `'Economy'` (below 130).

Q210. Label each market date as `'Weekend'` or `'Weekday'`.

Q211. Label each market date as `'Snow Day'`, `'Rain Day'` or `'Clear'`. If a date ever had both flags set to 1, which label would it get? Why does the order of WHEN clauses matter?

Q212. Put each purchase into a bill tier: `'High'` (total > 500), `'Medium'` (200 to 500), `'Low'` (below 200).

Q213. What will this return for a booth priced at 170? Fix the logic.

```
SELECT booth_price,
       CASE WHEN booth_price > 100 THEN 'Mid'
            WHEN booth_price > 150 THEN 'High'
            ELSE 'Low' END AS tier
FROM booth;
```

Q214. Write a CASE on `booth` without an ELSE clause that only labels Large booths. What value do the other booths get?

Q215. Sort the products by size in the order **Small** → **Medium** → **Large** (not alphabetical), using CASE in ORDER BY. Break ties by product_id.

Q216. Using the first digit of `customer_zip`, assign a region: 0–3 → `'East'`, 4–6 → `'Central'`, 7–9 → `'West'`.

Q217. Classify each purchase by `transaction_time` into `'Morning'` (06:00–11:59), `'Afternoon'` (12:00–16:59), `'Evening'` (17:00–21:59) and `'Night'` (everything else).

Q218. Convert `transaction_time` into a 12-hour label such as `'10 AM'`, `'12 PM'`, `'1 PM'`, `'12 AM'`, using CASE with LEFT and arithmetic.

Q219. Write a searched CASE that maps `market_day` to `'Start of Week'` (Monday, Tuesday), `'Mid Week'` (Wednesday, Thursday) and `'Week End'` (the rest). Then rewrite it as a simple CASE. When is each form better?

Q220. Data check: flag each booth `'OK'` if its description matches its size (Large ↔ 10x10, Medium ↔ 8x8, Small ↔ 6x6), otherwise `'Mismatch'`.

Q221. Product sizes should follow the cycle Small, Medium, Large by `product_id` (1 → Small, 2 → Medium, 3 → Large, 4 → Small, ...). Use CASE with `%` to work out the expected size, and return only the products whose actual size is different.

Q222. From `vendor_inventory`, add a `stock_status` column (`'Low'` < 50, `'Moderate'` 50–149, `'High'` ≥ 150) and a `restock_needed` column (`'Yes'` or `'No'`).

Q223. Label each market date's feel with a nested CASE: if it snowed, `'Snowy & Cold'` when min < 30, otherwise `'Snowy'`. If it didn't snow, `'Cold'` when min < 30, otherwise `'Mild'`.

Q224. Customers who buy 10 or more units get a 10% discount. Return only the purchases whose **final** amount (after the discount, if any) is above 300. Use CASE inside the WHERE clause.

Q225. Apply quantity slabs to `customer_purchases`: 1–5 units no discount, 6–10 units 5%, 11–15 units 10%, 16+ units 15%. Show the discount %, discount amount and final amount, rounded to 2 decimals.

Q226. Format customer names with a CASE: if the first name has 3 letters or fewer, show the full name. Otherwise show the first initial and the last name (e.g. `A. Johnson`).

Q227. Map `vendor_type` with a simple CASE: `'Type A'` → `'Organic'`, `'Type B'` → `'Conventional'`, anything else → `'Unclassified'`.

Q228. What happens when the branches of a CASE return different data types, e.g. `CASE WHEN booth_price > 150 THEN 'High' ELSE 0 END`? What type does the result column get?

Q229. Write a nested CASE on `booth`: first split by `booth_size`, then within each size label the booth `'Costly'` or `'Affordable'` depending on whether its price is above 150.

Q230. Using CASE with IN, label market days as `'Peak'` (Friday, Saturday, Sunday) or `'Off-Peak'`, and show only Peak days that had no snow.

Q231. In one SELECT on `market_date_info`, create three 1/0 flag columns: `is_weekend`, `is_snow_day` and `is_below_freezing`.

Q232. Sort booths so that all Large booths (`'10x10'`) come first in **descending** price order, followed by all other booths in ascending booth number. Use CASE in ORDER BY.

Q233. You created a CASE column called `bill_tier` in the SELECT list. Why does `WHERE bill_tier = 'High'` fail? Write a working query that returns only the 'High' bills without using a subquery.

Q234. Flag each customer as `'Check Zip'` if the zip code starts with `'0'` or is not exactly 5 characters long, otherwise `'Valid'`.

Q235. Label each market date `'First Half'` (day 1–15) or `'Second Half'` (day 16 onwards), using RIGHT on the date.

Q236. Add a 5% night surcharge to purchases made before 06:00 or after 21:00. Show `total_amount`, a `surcharge` column (0 when none applies) and `final_amount`, rounded to 2 decimals.

Section 13: Scenario-Based Questions

Q237. Data check on `customer_purchases`: find the rows where `product_id`, `vendor_id` and `customer_id` are **not** all equal to each other.

Q238. Create a username for every customer: the first 3 letters of the first name, the last 3 letters of the last name and the last 2 digits of the zip, all in lowercase (e.g. `alison45`).

Q239. For Type A vendors with `vendor_id` between 5 and 15, produce the sentence **"Vendor 7 (Type A) is owned by E. Jones"**.

Q240. From `market_date_info`, create a week label in the format **"W01-2024"**, zero-padding the week number to 2 digits. Show only distinct labels.

Q241. From `vendor_inventory`, find the rows where `vendor_id` is odd and the original price is between 20 and 30. Add a CASE column `value_band` based on inventory value: `'Small'` < 1000, `'Medium'` 1000–3000, `'Large'` > 3000.

Q242. For all booths except Medium booths, split `booth_size` into width and length, calculate the area and the price per sq ft (2 decimals), and label the booth `'Good Value'` if the price per sq ft is below 2, otherwise `'Pricey'`.

Q243. Show each customer's name masked as **"A J**"**: the first letter of each name followed by stars for the remaining characters.

Q244. Create a receipt line for each purchase in this format: `Receipt #0001 | 2024-01-01 10:00 | Qty 2 x $20.00 = $40.00` The receipt number is `customer_id` zero-padded to 4 digits, and the time shows hours and minutes only.

Q245. Find the customers whose full name, with no space, has an even number of characters **and** whose zip starts with a digit from 1 to 5.

Q246. Using the number extracted from `product_name`, find the products whose number is odd and between 5 and 15, and whose size is not 'Small'.

Q247. Find the weekday market dates where the minimum temperature in Celsius, rounded to 0 decimals, is below 0. Show the date, the day, the rounded Celsius value and a label: `'Freezing Weekday'`.

Q248. Build an `owner_display` column in the format **"DAVIS, F."** (last name in uppercase, comma, first initial). Include only vendors whose owner's last name contains a double letter or ends with `'s'`.

Q249. Find the purchases made in the second market week (7 Jan to 13 Jan 2024) between 3 PM and 9 PM. Show the day of the month and the hour (both extracted with string functions) and the total rounded to the nearest dollar.

Q250. Write the logical execution order of a query that uses FROM, WHERE, SELECT, DISTINCT and ORDER BY. Use it to explain why a SELECT alias works in ORDER BY but not in WHERE, with an example on `vendor_inventory`.